Selecting Manual Regression Test Cases Automatically using Trace Link Recovery and Change Coverage *

Sebastian Eder,
Benedikt Hauptmann, Maximilian Junker
Technische Universität München, Germany
{eders,hauptmab,junkerm}@in.tum.de

Rudolf Vaas, Karl-Heinz Prommer
Munich Re, Germany
{rvaas,hprommer}@munichre.com

ABSTRACT

Regression tests ensure that existing functionality is not impaired by changes to an existing software system. However, executing complete test suites often takes much time. Therefore, a subset of tests has to be found that is sufficient to validate whether the system under test is still valid after it has been changed. This test case selection is especially important if regression tests are executed manually, since manual execution is time intensive and costly.

To select manual test cases, many regression testing techniques exist that aim on achieving coverage of changed or even new code. Many of these techniques base on coverage data from prior test runs or logical properties such as annotated pre and post conditions in the source code. However, coverage information becomes outdated if a system is changed extensively. Also annotated logical properties are often not available in industrial software systems.

We present an approach for test selection that is based on static analyses of the test suite and the system's source code. We combine *trace link recovery* using latent semantic indexing with the metric *change coverage*, which assesses the coverage of source code changes. The proposed approach works automatically without the need to execute tests beforehand or annotate source code. Furthermore, we present a first evaluation of the approach.

Categories and Subject Descriptors

K.6.3 [**Management of Computing and Information Systems**]: Software Management; D.2.4 [**Software Engineering**]: Testing and Debugging

General Terms

Software Maintenance, Coverage Testing, Tracing

Keywords

Regression tests, test selection, manual system tests, test coverage, trace link recovery, software maintenance

1. INTRODUCTION

Business information systems often run and are maintained for up to two or three decades. For these long living systems, existing functionality must be preserved in spite of modifications. Therefore, regression testing detects changes that break existing functionality. In the field of business information systems, regression testing is often done manually by using test cases written in natural language.

An example of a manual test case is illustrated in Figure 1: A test case is separated into different steps and every step contains an action the tester has to perform and the expected result the tester has to check.

Regression testing ensures that existing functionality of the system under maintenance is not impaired. However, due to the size of test suites and limited resources, just subsets of test suites can be executed. Thus, the selection strategy is crucial for the effectiveness of regression testing. During maintenance, when a system is changed and tested afterwards, code changes are possibly missed by tests, which leads to field bugs. As we already showed in previous work [8], the rate of field bugs is higher in untested and changed code than in other code regions and therefore, it is more important to test changed methods. Based on these results, we now present an approach to regression test case selection, which targets changed code, for manual system tests which are written in natural language.

	Action	Expected result
1	Start the system	System is ready
2	Login using a valid username and password	Systems displays a message that you are logged in successfully
...		
6	Enter data for calculation	System shows dialog for exporting the results
7	Export results to Excel	Excel file is created
8	Open files	Data is correct

Figure 1: Example: Manual test case.

*This work was performed within the Software Campus project *ANSE*; it was funded by the German Federal Ministry of Education and Research (BMBF) under grant no. 01IS12057. The authors assume responsibility for the content.

AST'14, May 31 – June 1, 2014, Hyderabad, India
Copyright 2014 ACM 978-1-4503-2858-6/14/05...$15.00
http://dx.doi.org/10.1145/2593501.2593506

Problem: Existing test case selection approaches often rely on code coverage data of tests, or on logical properties such as pre and post conditions annotated in source code [10] used to predict which code regions might be executed by a test. However, these approaches are not generally applicable because coverage data from prior test runs becomes outdated if the system under test underlies extensive changes. Furthermore, logical properties are usually not contained in industrial software systems, because they are costly and difficult to create and maintain. This renders existing regression test selection techniques difficult to apply in real world scenarios.

Contribution: We propose an approach based on static analysis to identify regression tests that cover changed code. We do not rely on coverage data nor on annotations in source code, but present a fully automated approach based on *Latent Semantic Analysis* [7] to relate manual tests to source code, recovering trace links between both. With these trace links, we select test cases that have to be executed to test a certain piece of code. We use the metric *change coverage* to identify what parts of a system have been changed and are therefore subject to regression testing. Combining trace link recovery and change coverage enables identifying test cases as regression tests to test the changed code. Goal of the approach is, given a set of methods, that were changed, but not tested, to select test cases that execute methods. Additionally, we provide initial results of an evaluation of our technique, showing that the approach performs well in selecting relevant tests.

2. RELATED WORK

Selective regression testing or regression test selection techniques identify test cases based on changes of the system under test. There are several approaches to this topic [3, 4, 12, 22, 5]. However, these approaches mainly use code coverage information from prior test runs to identify which test case executes which code regions. Furthermore, most approaches focus on being *safe*, in terms of being capable of uncovering the same bugs the complete test suite could also reveal. We focus on selecting at least one test case that executes changed but untested methods. Additionally, these approaches tend to recommend many test cases which might be too much for an application in practice, especially if the tests have to be executed manually [15]. In contrast, we only select test cases that target changed code that has not been covered by tests since their modification and use only fully automated static analyses and therefore do not require the test suite to be executed to identify relevant test cases nor do we rely on logical properties of test cases or the system.

Test case prioritization orders test cases on their likeliness to detect bugs based on coverage metrics such as coverage of changes [20, 23]. Our work is related to the field of test case prioritization, since our approach can be used for this task too. As noticed in [8], changed methods indeed cause more field bugs than unchanged methods. Therefore, we suggest testing untested methods and prioritizing test cases covering these methods high.

Trace link recovery and feature location focus on uncovering relations between different artifacts such as locating higher level features in source code. Many approaches borrow from the fields of information retrieval and natural language processing. Cleland-Huang et al. [6] give an overview on the applicability of trace link recovery and proposes best practices. To the best of our knowledge, there is no approach using traceability link recovery for test case selection, which is the main contribution of this paper.

There are some approaches that target the recovery of trace links between natural language documents in software engineering (e.g. [16, 13]). Besides other algorithms [18], these approaches often use the Vector Space Model or LSI for retrieving links between documents. Falessiet al. [11] report on using and calibrating retrieval algorithms. We use this knowledge to calibrate the presented approach uses and rely on best practices presented by other researchers.

Much work has been done on recovering links between code and design artifacts. Especially Zhao et al. [24] present an approach very similar to our approach: Technically, they also use the call graph of a system to find representative code regions and propagate the tracing results through the call graph (we illustrate our approach in detail in Section 3). They use this technique to relate requirements to code regions and validate their approach based on two open source systems. As they achieve very good results (ca. 100% precision and ca. 95% recall) in linking code regions to requirements, we follow their work. However, they note that requirements have to be well designed to match to code. We use a very similar approach, but evaluate it on real world test cases from industry, rather than requirements documents.

Similarly to Zhao et al. [24], an approach using LSI and the static call graph of the software system under maintenance is presented by Charrada et al. [2]. They use traceability link recovery to detect outdated requirements based on source code changes, by extracting concepts from the source code that are relevant for requirements. We conform to their approach by also using the call graph, but do not consider requirements, but link source code methods to regression test for selection.

The selection properties to consider when tracing from source code to test cases or other artifacts is crucial. In [1], a study about which properties of the source code to take into account is given. They suggest using class names for tracing, but also show that using method names produces good results. Encouraged by this, we use method signatures, since tracing on class level is too coarse in our context.

Lucia et al. [17] introduce an approach facilitating tracing between code and test cases. They also use LSI as technique for comparing documents, but do not give details about how code is preprocessed prior to comparing it to test cases. Furthermore, it remains unclear, whether the test cases are written in natural language. Our approach is dedicated to trace from source code to test cases written in natural language for manual testing.

3. APPROACH

In this section, we introduce our approach used for regression test case selection. After giving a short overview of the approach, we explain the input to our approach and afterwards illustrate the main parts: Change Coverage and Trace Link Recovery using latent semantic indexing and the static call graph of the system under consideration.

Overview: To select test cases, we first seek code areas with methods that were changed but not tested as presented in [8]. We then run the trace link recovery algorithm (based on LSI) to find test cases that might cover these gaps. But as this approach is based on similarities, it *suggests* test cases.

3.1 Starting Point

The algorithm uses test case descriptions in natural language. We consider a test case as a plain text document written in natural language.

Furthermore, the approach expects a software system as input. Currently, we only support the programming language C#. The software system is given to the algorithm as compiled Intermediate Language (IL) code[1]. Additionally, the approach uses different versions of the same software system under maintenance to detect changes to the source code. Changes are considered only at method level (a method was changed or not). We consider a method as changed if its IL code differs from the previous version.

For calculating change coverage (see the following section), we use profiling data from an ephemeral profiler [21] as in our earlier work [8, 9]. The profiler does not record how often a method was called, just whether it was called during a fixed time interval. This data suffices to gain valuable insights into the test coverage of a system under maintenance.

With the data about method changes and the coverage information on method level, change coverage is calculated.

3.2 Change Coverage

To select code regions for which we select test cases, we use *change coverage*, since it quantifies the amount of changes covered by tests. Change coverage is calculated by the fraction number of methods that were changed and tested afterwards and the number of all changed methods (as shown in Equation 1). A detailed description can be found in [8]. The same study shows that collecting coverage information at method level is fine grained enough to gain insights into code coverage in practice, but does not produce too much performance overhead for practical application.

$$\text{change coverage} = \frac{\#\text{methods changed-and-tested}}{\#\text{methods changed}} \quad (1)$$

A change coverage of 1 means that all methods which have been changed have been tested after their last change. On the contrary, a coverage of 0 indicates that none of the changed methods has been covered by a test.

We perceive change coverage as a simple metric that is easy to understand. As reported earlier [8], this helped transferring it into practice.

3.3 Latent Semantic Indexing

As our approach to trace link recovery is based on Latent Semantic Indexing (LSI) [7], we explain it in this section. LSI is a technique to compute for each pair from a corpus of documents how similar they are to each other. The measure for the similarity is a value between zero and one, where one means identical [2] .Compared to other techniques LSI identifies groups of words that belong to a common concept (e.g. car and automobile belong to the same concept), which makes the computation of similarities more precise. Schematically LSI works as follows (and as illustrated in Figure 2): First, every document is represented by a vector in the space of words. Each entry in this vector denotes the weight that this word has with respect to this document (e.g. how often it occurs). Thus the whole set of documents is represented by a term-document matrix. Now LSI applies a procedure called Singular Value Decomposition (SVD). The result of SVD is a smaller matrix where terms are replaced by concepts, thus each document is now represented by a vector in the space of concepts. The similarity between two documents can then be calculated as the distance of the correspondent vectors.

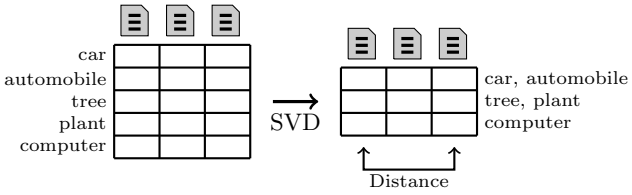

Figure 2: Schematic working of LSI.

3.4 Trace Link Recovery

The procedure for trace link recovery is divided into several steps, as illustrated in Figure 3. These steps are explained in detail below.

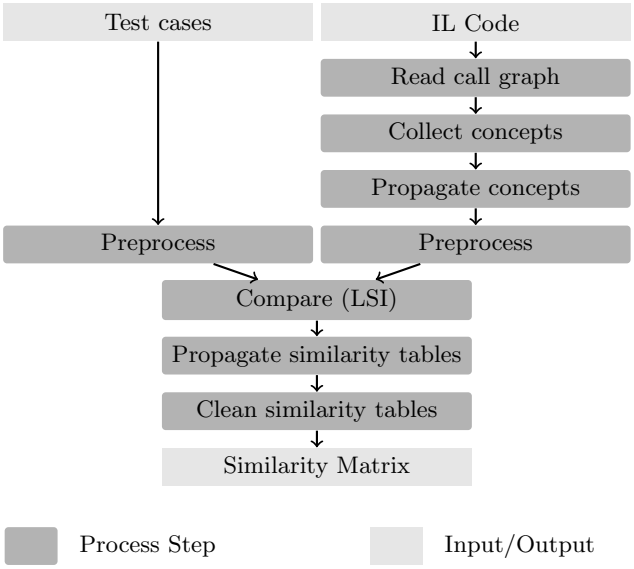

Figure 3: Overview of the approach.

Preprocessing of test cases consists of stemming and stop word removal[3]. Although stemming is theoretically not necessary when using LSI since the model recognizes synonyms given a dataset of sufficient size [14]. However, the size of the test suites we analyzed did not suffice to correctly classify synonyms (in the resulting vector space, two words with the same meaning are too far away from each other affecting the results in a negative way). Therefore, we use stemming to ensure every word is reduced to its normal form, expecting that stemming improves the results [14].

Transferring source code methods into documents: We divide this task in four steps:

Read call graph: The static call graph is retrieved directly from the intermediate language code (IL code).

[1]Call graph dependencies are easier to analyze in IL code.

[2]In the remainder of the paper, documents do not have to be identical, as we use the results from LSI as a measure of similarity.

[3]We used the stemming approach introduced by Porter [19].

Collect concepts: We rely on the common convention that identifiers are denoted in camelCase or different concepts are separated by underscores or dashes. With this assumption, we can split methods signatures into the concepts they contain. For example the method `getRatingAgency()` is transferred into the list [get, rating, agency].

Propagate concepts: We extract the concepts from the signatures of all methods that are reachable in the static call graph from the method under consideration. If, for example, the method `getNameOfAgency()` is reachable from `getRatingAgency()`, the resulting list of concepts is [get, rating, agency, get, name, of, agency]. With this technique, we also get information for methods that are entry points to the system.

Preprocess: On this list, we perform stemming and stop word removal resulting in the list of concepts [get, rate, agenc, get, name, agenc][4]. Note that the same stemming is performed on test cases resulting in the same stemmed terms occurring in the test cases.

We represent the result of the preprocessing of methods as a set of text files, one per method, containing all the concepts of the method itself, and the concepts of all methods that are reachable from this method. These files do not contain correct natural language, but suffice to use it with LSI.

Compare: After preprocessing is finished, we compare the documents resulting from methods and test cases based on LSI. The outcome is a matrix containing the values that indicate how similar a test case is to a method. The LSI comparison is the most time consuming task in our approach.

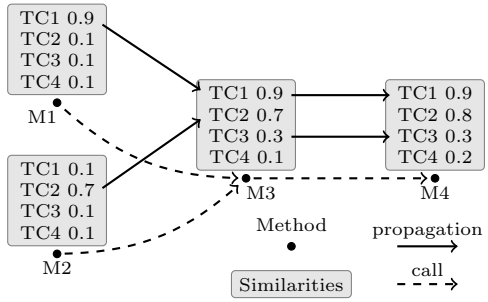

Figure 4: Example: Propagation of similarities.

Propagate similarity tables: Each system contains methods which are not similar to any test case, and therefore cannot be linked[5]. However, using the call graph, we know how to reach these methods. Therefore, we traverse along the call graph reversely to find methods which we can relate to tests with more confidence and propagate them back along the call graph. The result is that every method is annotated with the maximum similarity values for every test case from all methods it can be reached from in the call graph. Figure 4 shows an example of this propagation. With this procedure, we find tests cases for methods that are not mentioned in test cases directly.

Clean similarity tables: To this end, we calculated the similarities between every method and every test case. Since we only want to select relevant test cases, we keep only the elements in the list of similarities of a method where we expect that test case to execute this method. This is done as follows: Given a list of similarity values, sorted in descending order, we cut the list at the point where the biggest distance between two consecutive similarity values occurs. This is the same algorithm as used in [24]. Table 1 shows an example of the cutting algorithm: Between the test cases TC1 and TC2, the distance of the similarity values is 0.5, whereas the distance between TC3 and TC1, and TC2 and TC4 is only 0.1. Therefore, the list is cut between TC1 and TC2, since the distance of similarity values there is the biggest.

[4]"agenc" and "rate" result from stemming, and "of" is removed as it is a stop word.

[5]For example, logging facilities that are not mentioned in the test cases.

Table 1: Example for cutting similarity lists.

Test Case	Similarity
TC3	0.9
TC1	0.8
TC2	0.3
TC4	0.2

(a) Method M4 (before)

Test Case	Similarity
TC3	0.9
TC1	0.8

(b) Method M4 (after)

The result of the algorithm for trace link recovery is a matrix containing similarity values denoting the similarity of all methods to all test cases. An example of a similarity matrix is illustrated in Table 2, where TC1 to TC4 are test cases and M1 to M5 are source code methods.

Table 2: Example similarity matrix.

	M1	M2	M3	M4	M5
TC1	0.9	0.0	0.9	0.9	0.9
TC2	0.0	0.7	0.7	0.8	0.7
TC3	0.0	0.0	0.0	0.0	0.0
TC4	0.0	0.0	0.0	0.0	0.8

4. INITIAL EVALUATION

For evaluation, we conducted a case study in an industrial environment with which we answer the following questions:

RQ1 How accurate is the approach to trace link recovery? This question assesses for how many methods we find test cases that execute the methods.

RQ2 What is the influence of characteristics of test cases on the approach? With this question we measure the influence of characteristics of test cases such as the verdict of test cases and the number of steps that test functionality specific to the system on the accuracy of our approach.

4.1 Study Object

System under test: We perform the study on a business information system at Munich Re, which is one of the world's leading reinsurance companies with more than 47,000 employees in reinsurance and primary insurance worldwide. For their insurance business, they develop a variety of custom software systems. The business information system analyzed in this study implements damage prediction functionality and supports ca. 150 expert users in over ten countries. Table 3 illustrates the study object's main characteristics.

We chose this system as study object for several reasons: The system has been successfully in use for ten years and is still actively used and maintained. Moreover, it is a web

Table 3: Study object: System under test.

Language	C#
Age (years)	10
Size (kLOC)	340
Size (#methods)	39398

application, thus offering a single point for coverage data collection and accessible for researchers outside Munich Re.

Test suite: For an initial evaluation, we selected four manual test cases written in natural language[6]. The test cases we selected cover between 1297 and 2300 methods, and 2711 methods in total. The execution of the test cases takes between 5 and 15 minutes. The characteristics of the test cases are illustrated in Table 4. Column *Steps* shows the number of steps (consisting of action and expected result) a test case has and the column *Methods* denotes the number of methods a test case executes.

Table 4: Study object: Test cases.

Test Case	Steps	Methods
TC1	5	2300
TC2	7	1763
TC3	5	1297
TC4	11	1602

4.2 Study Design and Data Collection

Setup phase: To have reference values for the evaluation, we execute all test cases manually on the system under test and measure the code coverage for each test case. This data will deal as an oracle for our evaluation. However, some parts of the code have not been executed at all since there were no test cases covering this code. Since we have no data for these parts, we focus on executed methods in this study.

This oracle enables us to compare the results from our approach with the actual coverage data.

Evaluation phase: We define a link suggested by our tool from a source code method to a test case as correct, if the method has been executed during the actual test run of the suggested test. We calculate the accuracy of our approach by counting how many suggested links of the methods we executed in the setup phase are correct.

For answering RQ1 (How accurate is the approach to trace link recovery?), we calculate the accuracy of links from source code methods to test cases as the ratio of correctly linked methods and all considered methods as shown in Equation 2.

$$\text{accuracy} = \frac{\#\text{correctly-linked}}{\#\text{considered-methods}} \tag{2}$$

We compare our approach to randomly guessing a test case for every method as a baseline. Furthermore, we apply Pearson's Chi-Squared Test to test the null hypothesis H_0: The presented approach produces no significantly different results as the baseline. We chose a p value of 0.01.

To answer RQ2 (What is the influence of characteristics of test cases on the approach?), following the links from test cases to source code methods, we investigate the influence of the test cases on the accuracy. We calculate in how many cases the links suggested by our tool from test cases to source code methods are correct. To measure the performance of our approach, we calculate its precision using Equation 3, where true positives are links between methods that are really executed by the linked test case and false positives are links that connect methods with test cases that do not execute the method.[7]

$$\text{precision} = \frac{\#\text{true-positives}}{\#\text{true-positives} + \#\text{false-positives}} \tag{3}$$

4.3 Results

Results for RQ1 (How accurate is the approach to trace link recovery?) The proposed approach suggests at least one test case for every method. As we have four test cases for evaluation, assuming that every method is executed by exactly one test case, guessing would yield a chance of $\frac{1}{4} = 25\%$ to hit the right test case. Since methods might be executed by more than one test case, we define the baseline for RQ1 higher: From our analyses we know that each method is executed by 2.56 test cases in average. Therefore randomly guessing yields a chance for guessing a correct test case of $\frac{2.56}{4} = 64\%$ as a baseline for our evaluation.

For the first research question, the results are presented in Table 5: We considered 2711 source code methods in total and linked 2444 methods with a test case that executes them. With 90% of all considered methods classified to a correct test case, we gain considerably better results than the baseline.

Table 5: Results: Methods.

Total	Correctly linked	Accuracy
2711	2444	90%

We link 1.75 test cases to a method in average with a variance of 0.60; 1237 methods are linked to one test case, 909 methods are linked to two test cases, 565 methods are linked to three test cases, and no method is linked to four test cases (this results from the design of the algorithm for cutting similarity lists).

For a statistical interpretation of the results, we observed 2444 methods as linked to a correct test case and 267 methods not linked to a correct test case. With the baseline of 64% of correctly linked test cases by randomly guessing (and 36% incorrectly linked methods), we expect 1740.5 methods to be linked to the correct test case and 970.5 methods to be incorrectly linked. The result of the Chi-Squared-Test ($\chi^2 = 764.3$) shows, that the null hypothesis H_0 can be

[6]For the study, execution traces of test cases are necessary. Since capturing execution traces of manual test cases in an industrial environment is a difficult task, because usually there is no profiling environment and test cases require deeper knowledge of the system under test, we had to face the trade-off between test cases from industry (but less) or artificial test cases (more). To gain more realistic results, we chose test cases from industry.

[7]Recall would measure whether our approach finds all methods executed by a test case or reversed, whether the approach finds all test cases for a method. Since the goal of the presented approach is to close gaps in change coverage and therefore executing methods that were changed, but not tested, it is enough to find one test case for every method, what we do *by design* and thus recall is not appropriate.

rejected and the approach outperforms randomly guessing with statistical significance ($p = 0.01$).

Results for RQ2 (What is the influence of characteristics of test cases on the approach?) For the second research question, we present the results for every test case in Table 6 focusing on links between test cases and methods. Column *Total links* contains the total number of all links suggested by our approach for the corresponding test case. Column *Correct links* contains the number of correct links from methods to the test case. The precision for every test case is contained in the second column indicating how many links of the suggested links for every test case are correct. Overall, we suggested 4750 links and 3619 (76%) of these links are correct.

Table 6: Results: Test cases.

Test Case	Precision	Total links	Correct links
TC1	95%	1558	1480
TC2	73%	852	618
TC3	68%	523	355
TC4	64%	1817	1166
Overall	76%	4750	3619

4.4 Discussion

RQ1: The use case of our approach is finding test cases that execute methods that were changed, but remained untested. Our approach suggests a correct test case in 90% of all considered methods while suggesting 1.75 test cases in average. These results indicate that the approach is suited for the use case, because first, the approach has a high hit rate and second, the approach limits the number of test cases to select.

RQ2: To enable a more detailed discussion, characteristics of the test cases we chose for the initial evaluation are presented in Table 7. Column *Interactions* shows the number of actions and checks that target the system under test (and not an external tool, like step 8 in the example in Figure 1) and therefore contain more words that are characteristic to the system. The value in brackets is the total number of steps contained in the test case (see Table 4). This influences our approach since the more characteristic words are contained in a test case the more terms of the test case are found in source code methods. Furthermore, we chose three test cases with a successful verdict and one failing test case. Column *Verdict* contains the outcome of the test case: Success means the test case was performed without revealing errors, and a failing test case produced errors. Table 7 also shows that we chose heterogeneous test cases in terms of verdict and system specific interactions for our evaluation.

We deliberately chose test cases containing not only interactions with the system under test but also with external tools, because we observed many test cases in the test suite we perform our analysis on exhibiting this characteristic. To gain more representative results for the system under consideration, it is thus necessary to also choose test cases with parts that are considering external systems.

Table 7: Study object: Test cases (extension).

Name	Interactions	Verdict
TC1	5 (5)	Success
TC2	7 (7)	Fail
TC3	2 (5)	Success
TC4	4 (11)	Success

We observe that the precision for TC1, which was executed without errors and only contains interactions regarding the system directly produces the best result considering precision (94%). This is not surprising, since this test case contains mainly words regarding the system and furthermore did not trigger error handling due to errors and thus did not execute program parts that are not mentioned in the test case. However, also TC2, which produced an error and could not be executed completely, can be related to methods with a precision of 73% (we successfully executed four steps in TC2). The remaining test cases T3 and T4 focus heavily on checks regarding data in an exported file. This leads to slightly worse results than we observe with TC1 and TC2.

These results indicate that the precision even rises when we consider test cases that contain more interactions specific to the system and are not failing.

Conjecture: The results show that out approach is able to suggest correct test cases for methods. Therefore, we consider it suitable for finding test cases for methods that are untested, but can be executed by an existing test case. Based on this, we formulate the following hypothesis which is subject to further investigation: Using trace link recovery based on LSI facilitates regression test selection for changed, but yet untested source code methods.

4.5 Threats to Validity

As we performed our trace recovery approach on only one system and four test cases, the results are hardly generalizable. However, we chose a system with a common size and complexity in business information systems. We chose heterogeneous test cases, which means successful and failing test cases and test cases with less or more system interaction and system specific text, to reduce the bias introduced by the small number of test cases.

It is possible that our collection of usage data for test runs did not collect all necessary data for the oracle. This can happen, for example, when caching mechanisms become active after one run of the system. We expect this effect to be small, since developers told us there are no such mechanisms. The collection procedure itself has shown to be working with sufficient accuracy in earlier research [9, 8].

Furthermore, the tracing algorithm might not be implemented correctly. We mitigated this threat by testing our implementation on other artefact types.

5. CONCLUSIONS AND FUTURE WORK

We presented an approach based on trace link recovery (using latent semantic indexing [7]) and change coverage [8] for regression test selection for test cases written in natural language and an initial evaluation of the approach. The approach yields the advantage that a changed program does not need to be executed again to gain enough information for selecting test cases or needs to be annotated again with logical properties to enable test case selection. The evaluation showed that the tracing mechanism works well for manual system tests: Our approach selects a correct test case for 90% of all considered source code methods and outperforms guessing randomly with statistical significance.

The repeated execution of a system under test to gain coverage data of test cases is not desirable, since this again is a task which needs resources that are usually missing, and this is why test case selection is performed in the first place. The annotation of logical properties to industrial software systems is difficult due to the size and complexity of these systems. Furthermore, modern programming languages, for example C# lack clear formal semantics, which is a prerequisite for gaining formal logical information about a system.

Our approach suggests test cases for every method. One effect of this is that if there is no test case covering the code region under consideration, our approach still might suggest a test case based on the concepts contained in the methods of the code region not covering the code. It is subject to future work to investigate effects of this behavior.

In our future work, we want to increase external validity by studying more manual test cases on different systems. Furthermore, we plan to use the trace link recovery approach for not only tracing to test cases, but also to other artifacts as requirements documents. Furthermore, we evaluate how different calibrations of the approach affect the results. Additionally, we plan to investigate the influence of missing test cases on our approach.

Another promising direction is the combination of the presented approach to trace link recovery with coverage based test case selection techniques (for example [5]). We expect to get good results using more complex techniques than change coverage for finding code that has to be tested. Furthermore, we are confident to gain better applicability of existing approaches, which rely on coverage data by applying our trace link recovery algorithm instead of using test coverage data.

6. ACKNOWLEDGMENTS

The authors thank Daniela Steidl, Jonas Eckhardt, Henning Femmer, and Elmar Jürgens for their helpful comments on this work.

7. REFERENCES

[1] G. Antoniol, B. Caprile, A. Potrich, and P. Tonella. Design-code traceability recovery: selecting the basic linkage properties. *Sci. Comput. Program.*, 40(2–3):213 – 234, 2001.

[2] E. Ben Charrada, A. Koziolek, and M. Glinz. Identifying outdated requirements based on source code changes. In *RE '2012*, 2012.

[3] J. Bible, G. Rothermel, and D. S. Rosenblum. A comparative study of coarse- and fine-grained safe regression test-selection techniques. *ACM Trans. Softw. Eng. Methodol.*, 10(2), 2001.

[4] V. Channakeshava, V. K. Shanbhag, A. Panigrahi, R. Sisodia, and S. Lakshmanan. Safe subset-regression test selection for managed code. In *ISEC '08*, 2008.

[5] Y.-F. Chen, D. Rosenblum, and K.-P. Vo. Testtube: a system for selective regression testing. In *ICSE '94*, 1994.

[6] J. Cleland-Huang, R. Settimi, E. Romanova, B. Berenbach, and S. Clark. Best practices for automated traceability. *Computer*, 40(6):27–35, 2007.

[7] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman. Indexing by latent semantic analysis. *J. Am. Soc. Inf. Sci. Technol.*, 41(6):391–407, 1990.

[8] S. Eder, B. Hauptmann, M. Junker, E. Juergens, R. Vaas, and K.-H. Prommer. Did we test our changes? assessing alignment between tests and development in practice. In *AST '2013*, 2013.

[9] S. Eder, M. Junker, E. Jurgens, B. Hauptmann, R. Vaas, and K. Prommer. How much does unused code matter for maintenance? In *ICSE '2012*, 2012.

[10] E. Engström, P. Runeson, and M. Skoglund. A systematic review on regression test selection techniques. *Inf. Softw. Technol.*, 52(1):14–30, 2010.

[11] D. Falessi, G. Cantone, and G. Canfora. Empirical principles and an industrial case study in retrieving equivalent requirements via natural language processing techniques. *IEEE Trans. Softw. Eng.*, 39(1):18–44, 2013.

[12] M. J. Harrold, J. A. Jones, T. Li, D. Liang, A. Orso, M. Pennings, S. Sinha, S. A. Spoon, and A. Gujarathi. Regression test selection for java software. *SIGPLAN Not.*, 36(11), 2001.

[13] J. H. Hayes, A. Dekhtyar, and S. K. Sundaram. Advancing candidate link generation for requirements tracing: The study of methods. *IEEE Trans. Softw. Eng.*, 32(1):4–19, 2006.

[14] D. Jiménez, E. Ferretti, V. Vidal, P. Rosso, and C. F. Enguix. The influence of semantics in ir using lsi and k-means clustering techniques. In *ISICT '03*, 2003.

[15] E. Juergens, B. Hummel, F. Deissenboeck, M. Feilkas, C. Schlögel, and A. Wübbeke. Regression test selection of manual system tests in practice. In *CSMR '11*, 2011.

[16] M. Lormans and A. van Deursen. Can lsi help reconstructing requirements traceability in design and test? In *CSMR '06*, 2006.

[17] A. D. Lucia, F. Fasano, R. Oliveto, and G. Tortora. Recovering traceability links in software artifact management systems using information retrieval methods. *ACM Trans. Softw. Eng. Methodol.*, 16(4), 2007.

[18] R. Oliveto, M. Gethers, D. Poshyvanyk, and A. De Lucia. On the equivalence of information retrieval methods for automated traceability link recovery. In *ICPC '2010*, 2010.

[19] M. F. Porter. An algorithm for suffix stripping. *Program: Electronic Library & Information Systems*, 40(3):211–218, 1980.

[20] G. Rothermel, R. Untch, C. Chu, and M. Harrold. Prioritizing test cases for regression testing. *Software Engineering, IEEE Transactions on*, 27(10), 2001.

[21] O. Traub, S. Schechter, and M. D. Smith. Ephemeral instrumentation for lightweight program profiling. Technical report, School of engineering and Applied Sciences, Harvard University, 2000.

[22] D. Willmor and S. M. Embury. A safe regression test selection technique for database-driven applications. In *ICSM '05*, 2005.

[23] W. Wong, J. Horgan, S. London, and H. Agrawal. A study of effective regression testing in practice. In *ISSRE '97*, 1997.

[24] W. Zhao, L. Zhang, Y. Liu, J. Sun, and F. Yang. SNIAFL: Towards a static noninteractive approach to feature location. *ACM Trans. Softw. Eng. Methodol.*, 15(2):195–226, 2006.